LEETCODE
&
FINAL CHALLENGE

WHAT IS LEETCODE?

- LeetCode is a website for coding practice
- It helps improve problem solving skills
- Commonly used by students and developers
- Good for interviews and competitions

WHY LEARN LEETCODE?

- Improve coding logic and algorithms
- Learn common problem patterns
- Practice for coding interviews
- Boost confidence in programming

PATTERN RECOGNITION

Pattern recognition is the ability to identify recurring structures or behaviors in problems and apply known techniques to solve them efficiently.

PATTERN RECOGNITION

1. Traffic lights

You recognize the repeating pattern: red → yellow → green. Knowing the pattern helps you predict what comes next.

2. Daily routine

Waking up, going to school/work, eating, and sleeping follow a pattern that helps you plan your day.

3. Phone unlock (PIN or pattern)

Recognizing the correct pattern allows access, while a wrong pattern fails.

4. Weather habits

If it gets dark and cloudy and usually rains after, you recognize the pattern and carry an umbrella.

TWO POINTERS

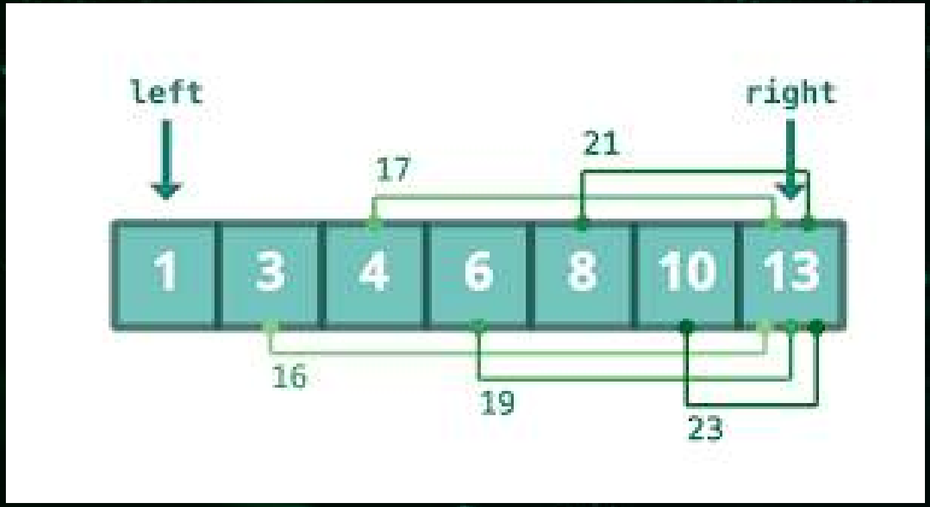
Two pointers is a technique where you use two indices to traverse a data structure usually an array or string from different positions to solve problems efficiently.

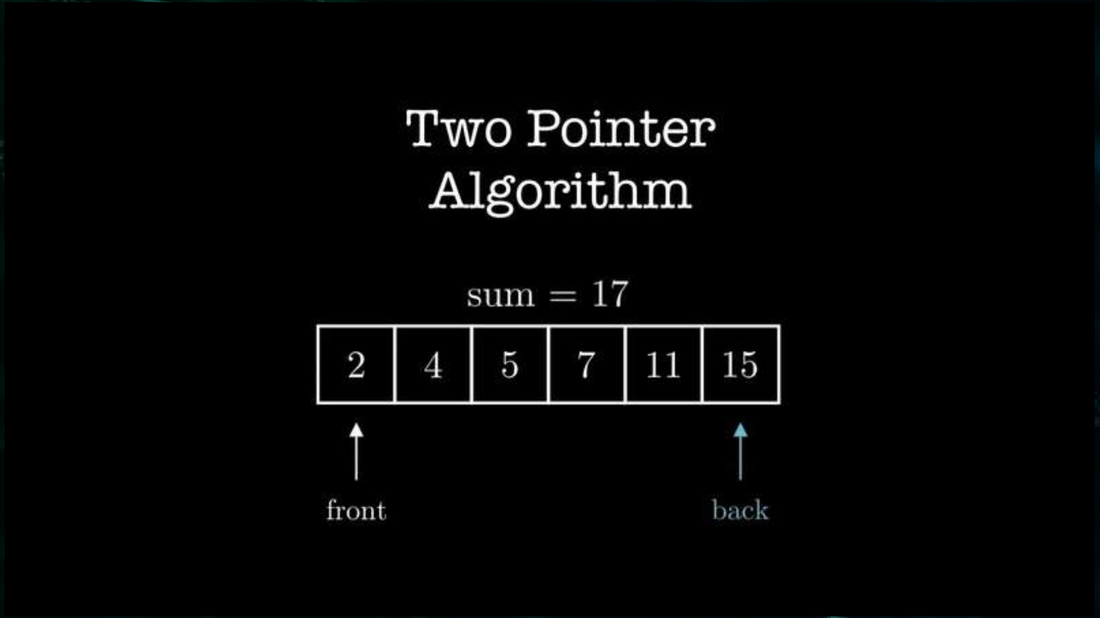
Examples:

1. Reversing a string by swapping characters from the start and end until the pointers meet.

2. Finding a pair with a target sum in a sorted array by moving left and right pointers inward.

TWO POINTERS





HASHMAPS

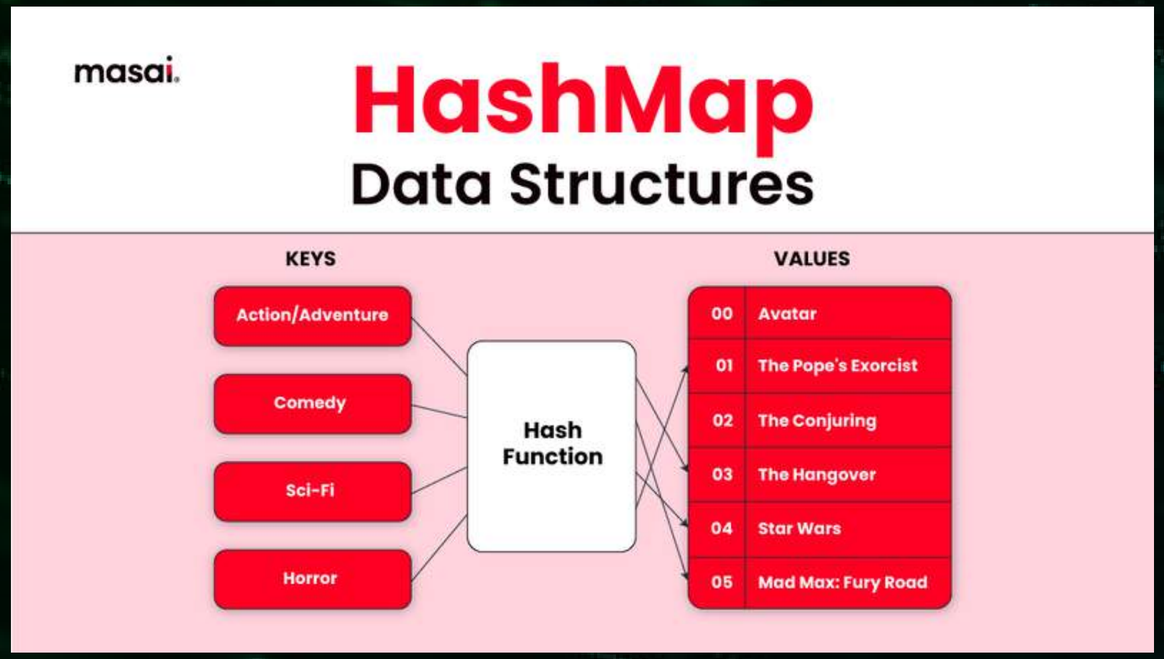
A hash map stores data as key-value pairs, allowing fast lookup, insertion, and deletion—typically in constant time.

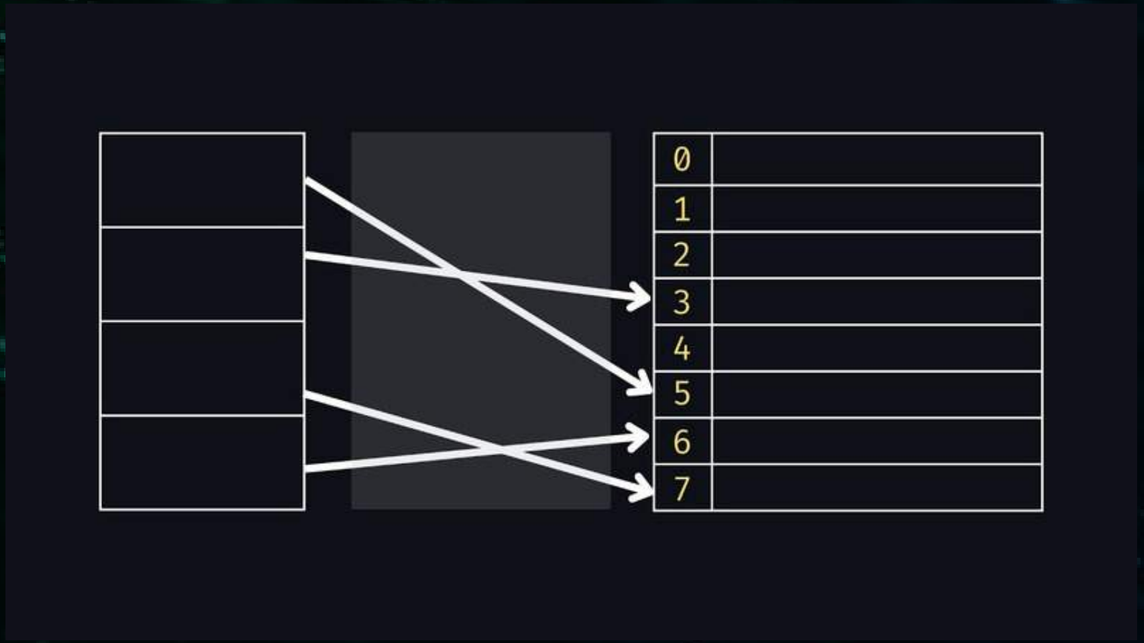
Examples:

1. Counting character frequency in a string (e.g., how many times each letter appears).

2. Two Sum problem, where you store numbers and their indices to quickly find a pair that adds up to a target.

HASHMAPS


masai.
HashMap
Data Structures
KEYS
VALUES
Action/Adventure
Comedy
Sci-Fi
Horror
Hash Function
00 Avatar
01 The Pope's Exorcist
02 The Conjuring
03 The Hangover
04 Star Wars
05 Mad Max: Fury Road


0
1
2
3
4
5
6
7

CASE STUDY 1: TWO SUM

Given an array of integers [**nums**] and an integer [target], return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

CASE STUDY 1: TWO SUM

Example 1:

	0	1	2	3
nums =	2	7	11	15

target = 9

output = [0, 1]

CASE STUDY 1: TWO SUM

Example 2:

	0	1	2
nums =	3	2	4

target = 6

output = [1, 2]

CASE STUDY 1: TWO SUM

Example 3:

	0	1	2	3	4
nums =	3	4	7	4	1

target = 8

output = [1, 3]

CASE STUDY 1: TWO SUM

BRUTE FORCE METHOD

```python
def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, nums):
            if nums[i] + nums[j] == target:
                return [i, j]
```

CASE STUDY 1: TWO SUM

Example 3:

	0	1	2	3	4
nums =	3	4	7	4	1
	i	j			

target = 8

is nums[i] + nums[j] == target?

CASE STUDY 1: TWO SUM

Example 3:

	0	1	2	3	4
nums =	3	4	7	4	1
	i		j		

target = 8

is nums[i] + nums[j] == target?

CASE STUDY 1: TWO SUM

Example 3:

	0	1	2	3	4
nums =	3	4	7	4	1
	i			j	

target = 8

is nums[i] + nums[j] == target?

CASE STUDY 1: TWO SUM

Example 3:

	0	1	2	3	4
nums =	3	4	7	4	1
	i				j

target = 8

is nums[i] + nums[j] == target?

CASE STUDY 1: TWO SUM

Example 3:

	0	1	2	3	4
nums =	3	4	7	4	1
		i	j		

target = 8

is nums[i] + nums[j] == target?

CASE STUDY 1: TWO SUM

Example 3:

	0	1	2	3	4
nums =	3	4	7	4	1
		i		j	

target = 8

is nums[i] + nums[j] == target?

if yes: return [i, j]

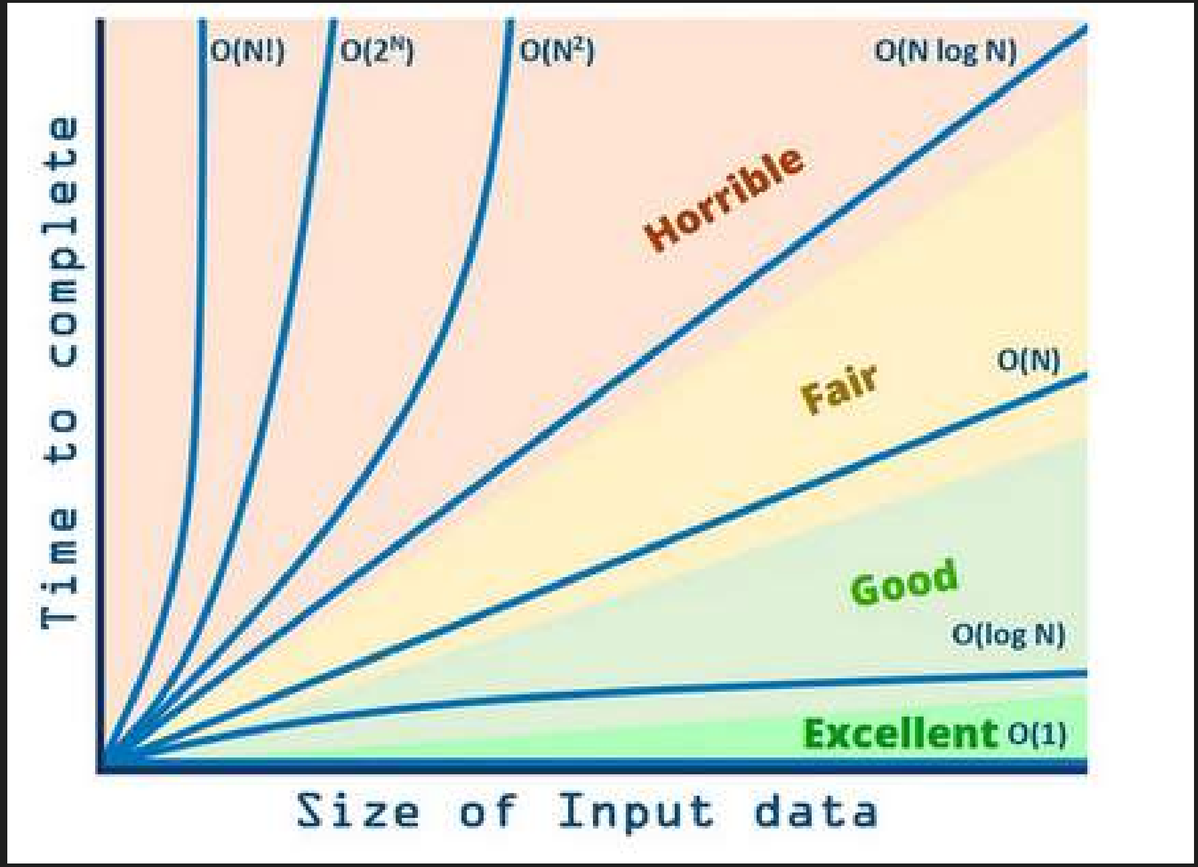
output = [1, 3]

OPTIMIZATION

Optimization is the process of improving a solution to make it more efficient by reducing time complexity, space usage, or both.

- Replacing a nested loop with a single loop to reduce time complexity from $O(n^2)$ to $O(n)$.
- Using a hash map instead of linear search to speed up lookups from $O(n)$ to $O(1)$.

COMPLEXITY CHART



CASE STUDY 1: TWO SUM

OPTIMIZED TWO SUM

```python
def twoSum(nums, target):
    hashmap = {}

    for index, num in enumerate(nums):
        if target - num in hashmap:
            return index, hashmap[target - num]
        hashmap[num] = index

        return None
```

CASE STUDY 1: TWO SUM

Example 3:

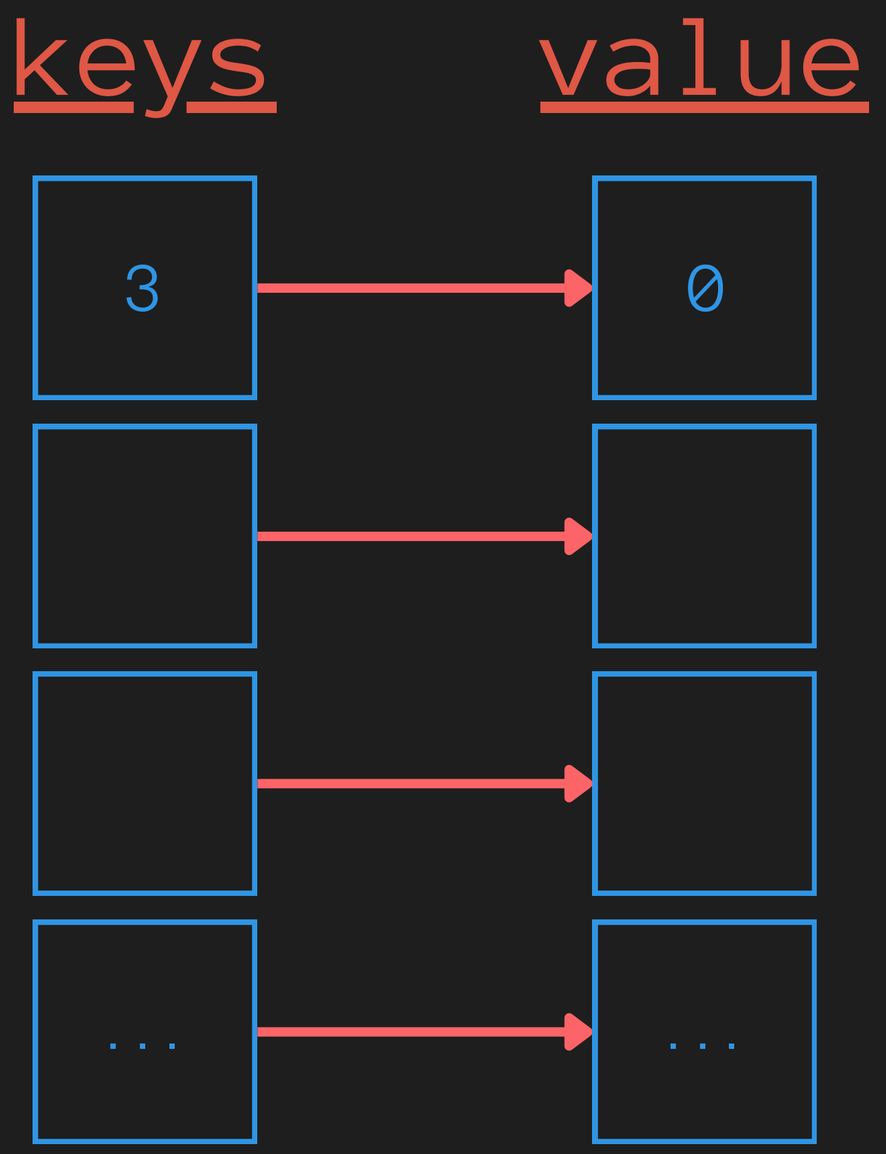
	0	1	2	3	4
nums =	3	4	7	4	1

target = 8

is target - current_num in keys?

if no:

store keys and value on hashmap



CASE STUDY 1: TWO SUM

Example 3:

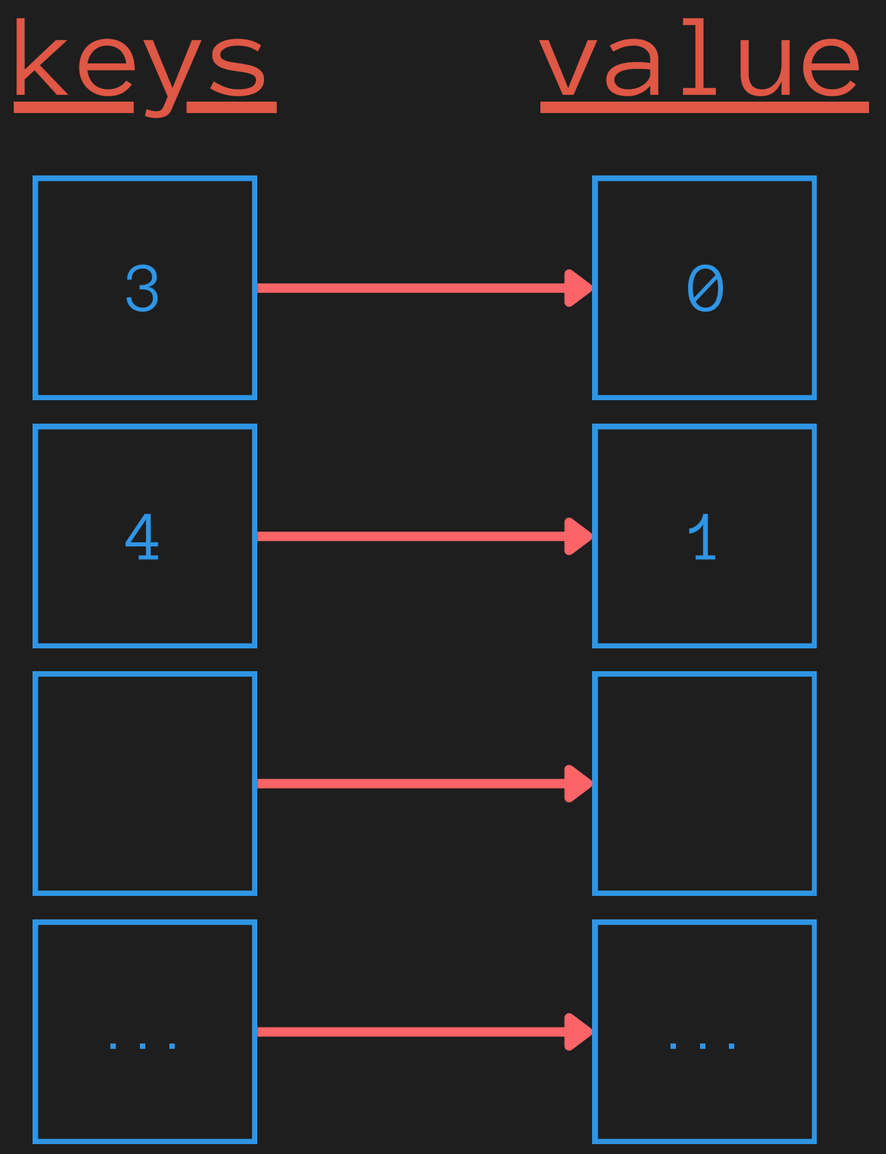
	0	1	2	3	4
nums =	3	4	7	4	1

target = 8

is target - current_num in keys?

if no:

store keys and value on hashmap



CASE STUDY 1: TWO SUM

Example 3:

	0	1	2	3	4
nums =	3	4	7	4	1

target = 8

is target - current_num in keys?

if no:

store keys and value on hashmap

keys	value
3	0
4	1
7	2
...	...

CASE STUDY 1: TWO SUM

Example 3:

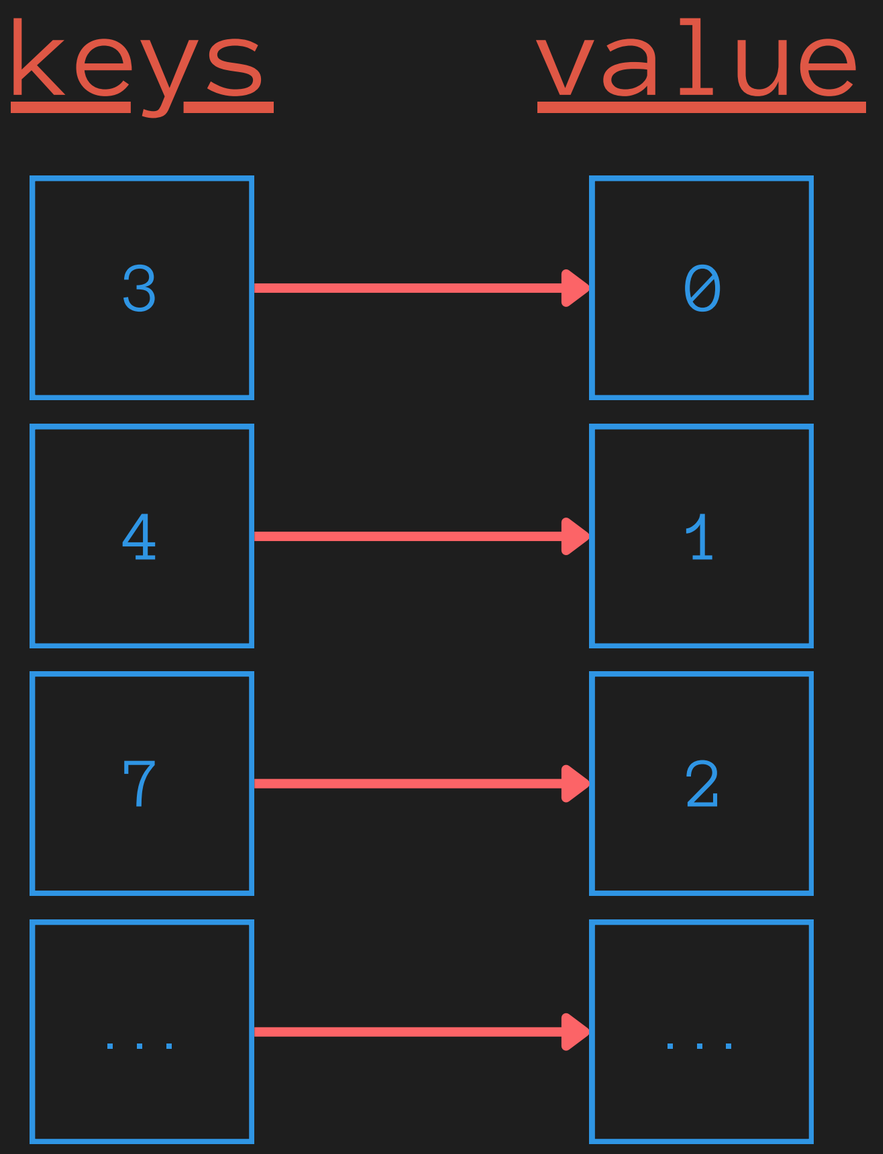
	0	1	2	3	4
nums =	3	4	7	4	1

target = 8

is target - current_num in keys?

```
if yes:
return index, hashmap[target - num]
```

output = [1, 3]



CASE STUDY 1: TWO SUM

Why Does a Hash Map Work in Two Sum?

For any two numbers a and b:

$$a + b = target$$

Rearrange it:

$$a = target - b$$

CASE STUDY 1: TWO SUM

Example:

how did we got this?

	0	1	2	3	4
nums =	3	4	7	4	1

target = 8

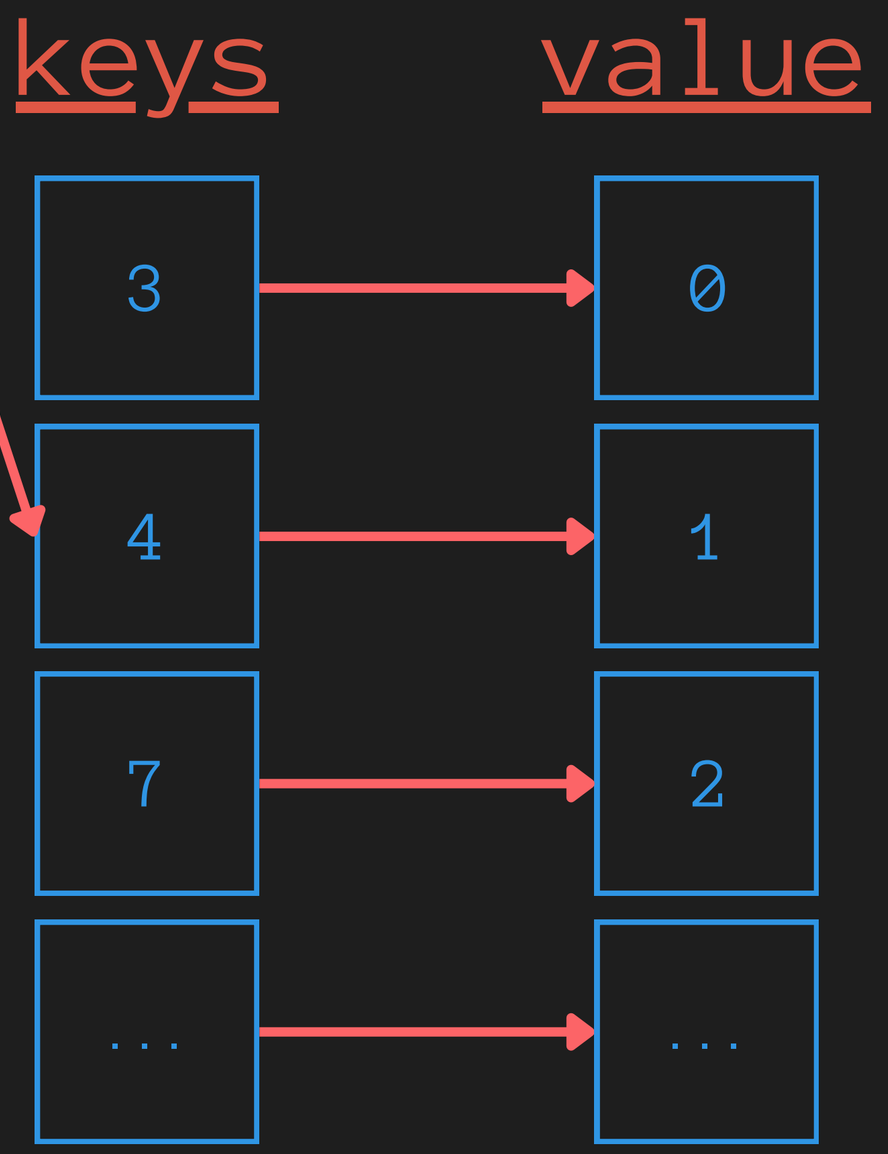
current_number = 4

needed_number = target - current_number

example:

n = 8 - 4

n = 4



CASE STUDY 2: PALINDROME NUMBER

Given an integer x, return true if x is a palindrome, and false otherwise.

Example 1:

Input: x = 121

Output: true

Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: x = -121

Output: false

Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

CASE STUDY 2: PALINDROME NUMBER

Given an integer x, return true if x is a palindrome, and false otherwise.

Example 3:

Input: x = 10

Output: false

Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

CASE STUDY 2: PALINDROME NUMBER

Given an integer x, return true if x is a palindrome, and false otherwise.

```python
def is_palindrome(num):
    original = num
    reversed_num = 0

    while num > 0:
        last_digit = num % 10
        reversed_num = reversed_num * 10 + last_digit
        num = num // 10

    return original == reversed_num
```

CASE STUDY 2: PALINDROME NUMBER

```
x = 121
reverse_number = 0
```

x	Last Digit	Reversed Number
121	121 % 10 = 1	0 * 10 + 1 = 1
12	12 % 10 = 2	1 * 10 + 2 = 12
1	1 % 10 = 1	12 * 10 + 1 = 121

CASE STUDY 3: VALID PALINDROME

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string s, return true if it is a palindrome, or false otherwise.

Example 1:

Input: s = "A man, a plan, a canal: Panama"

Output: true

Explanation: "amanaplanacanalpanama" is same if reverse therefor it is a palindrome.

CASE STUDY 3: VALID PALINDROME

Example 1:

Input: s = "race a car"

Output: false

Explanation: "raceacar" is not a palindrome.

Example 2:

Input: s = " "

Output: true

Explanation: s is an empty string "" after removing non-alphanumeric characters. Since an empty string reads the same forward and backward, it is a palindrome.

CASE STUDY 3: VALID PALINDROME

s =

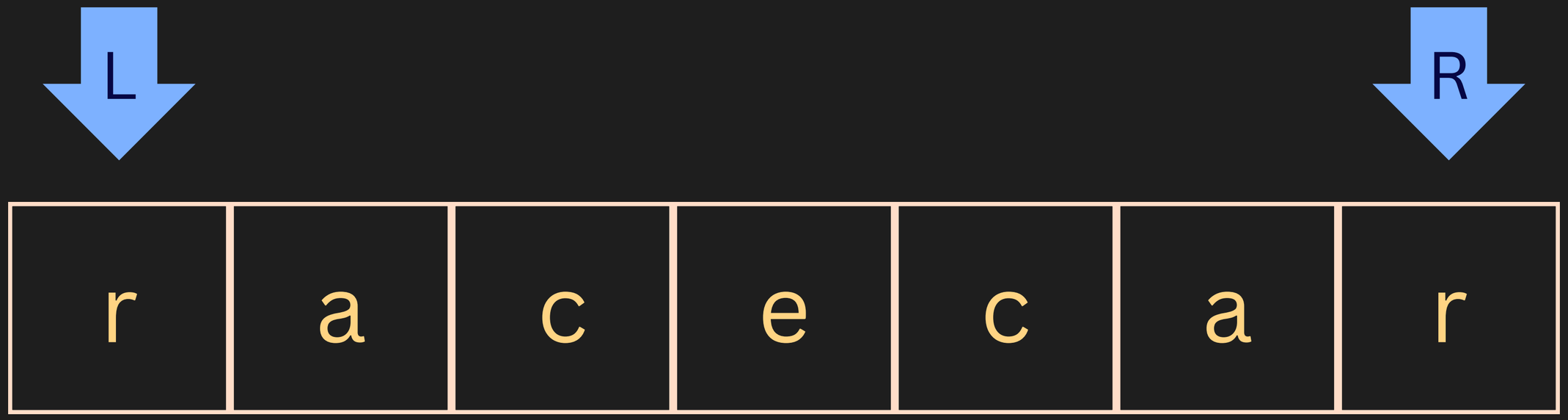
r	a	c	e	c	a	r

Is this a valid palindrome? Why?

How did you solve it mentally?

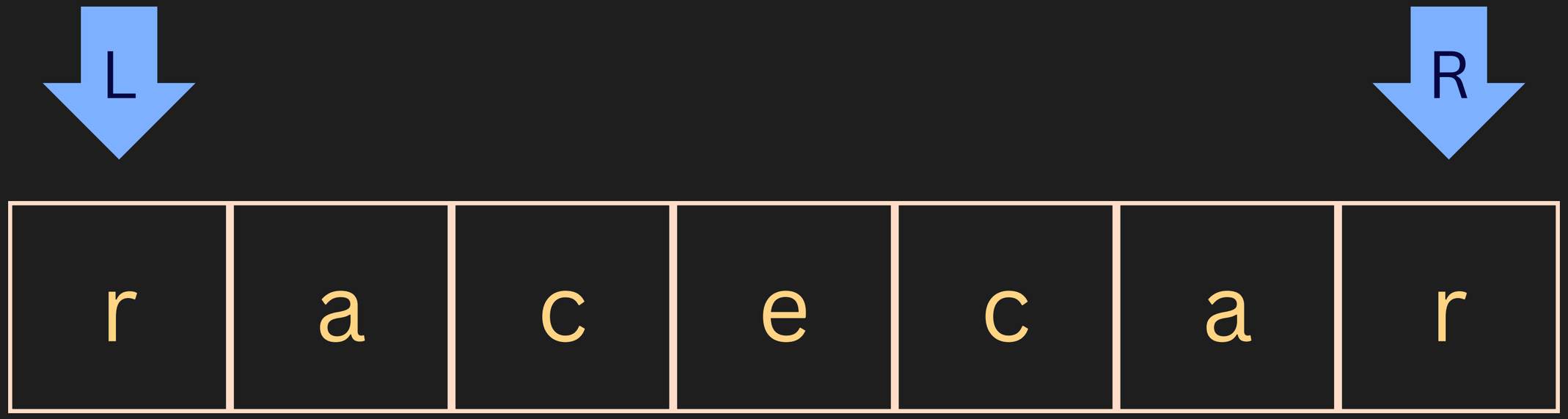
CASE STUDY 3: VALID PALINDROME

1. Remove spaces from the string and convert it to lowercase
2. Set one pointer at the start of the string and another at the end.
3. Compare the characters at both pointers.



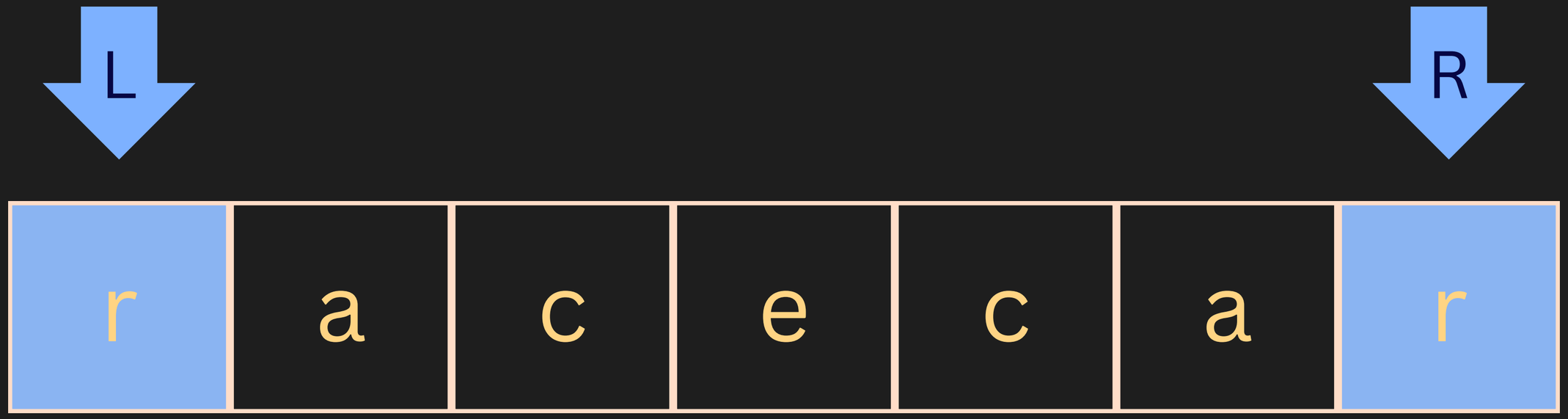
CASE STUDY 3: VALID PALINDROME

4. If the characters are different, the string is not a palindrome.
5. If the characters are the same move the pointers toward the center.
6. Repeat until the pointers meet.



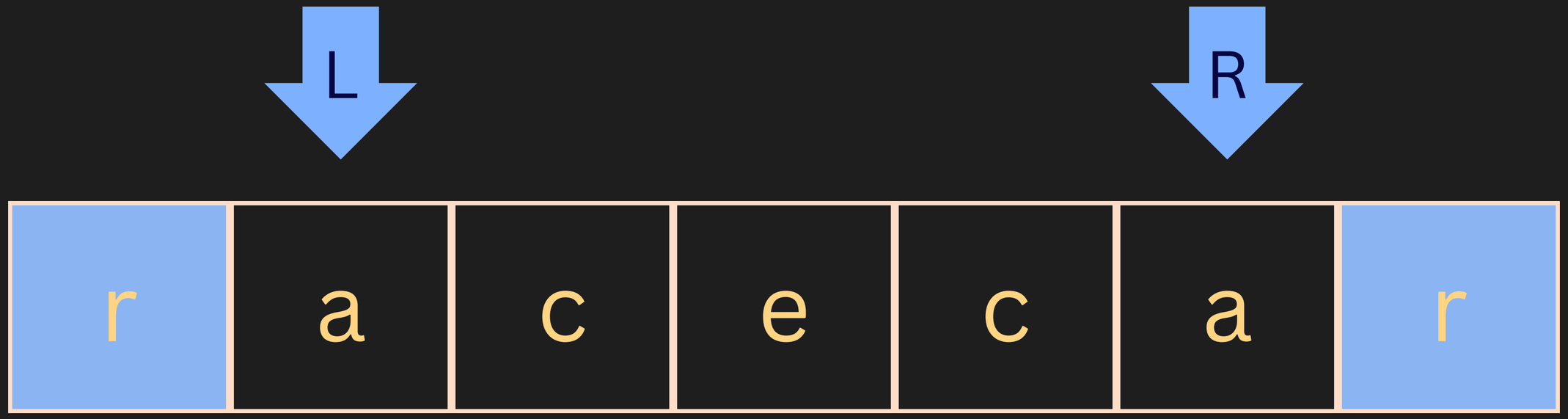
CASE STUDY 3: VALID PALINDROME

4. If the characters are different, the string is not a palindrome.
5. If the characters are the same move the pointers toward the center.
6. Repeat until the pointers meet.



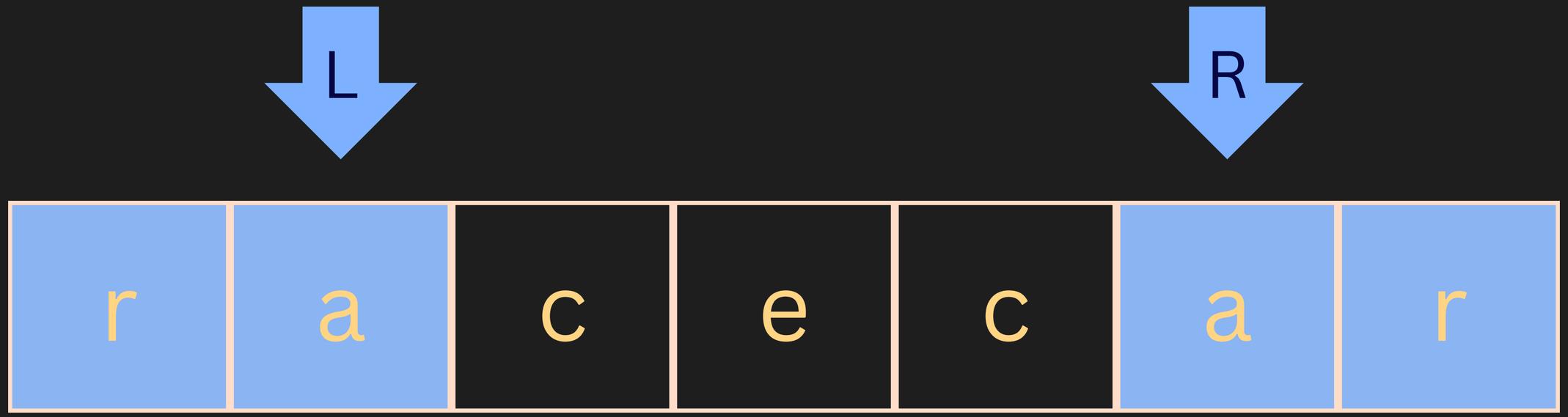
CASE STUDY 3: VALID PALINDROME

4. If the characters are different, the string is not a palindrome.
5. If the characters are the same move the pointers toward the center.
6. Repeat until the pointers meet.



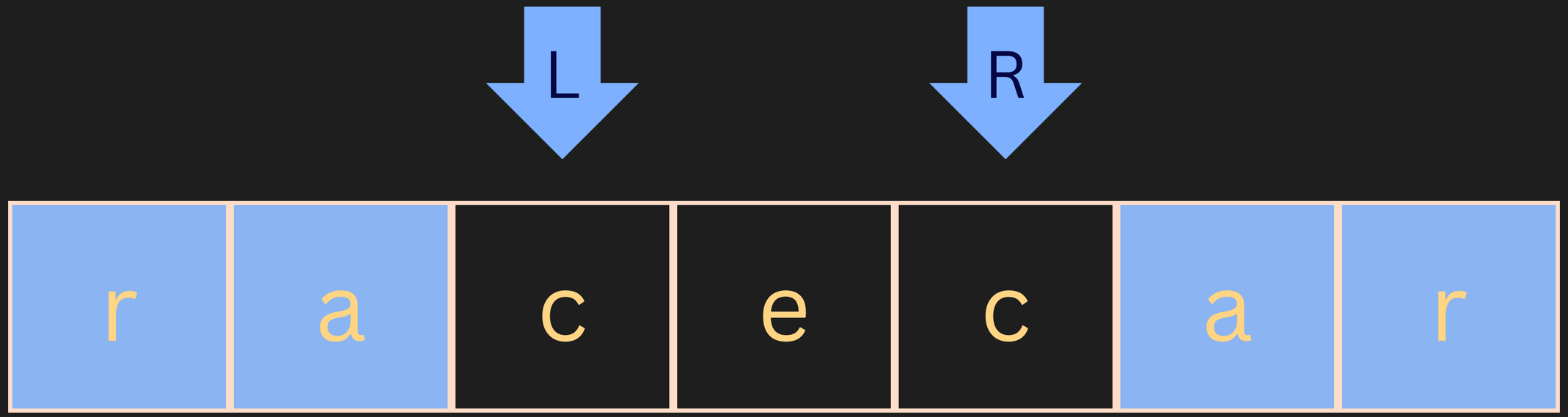
CASE STUDY 3: VALID PALINDROME

4. If the characters are different, the string is not a palindrome.
5. If the characters are the same move the pointers toward the center.
6. Repeat until the pointers meet.



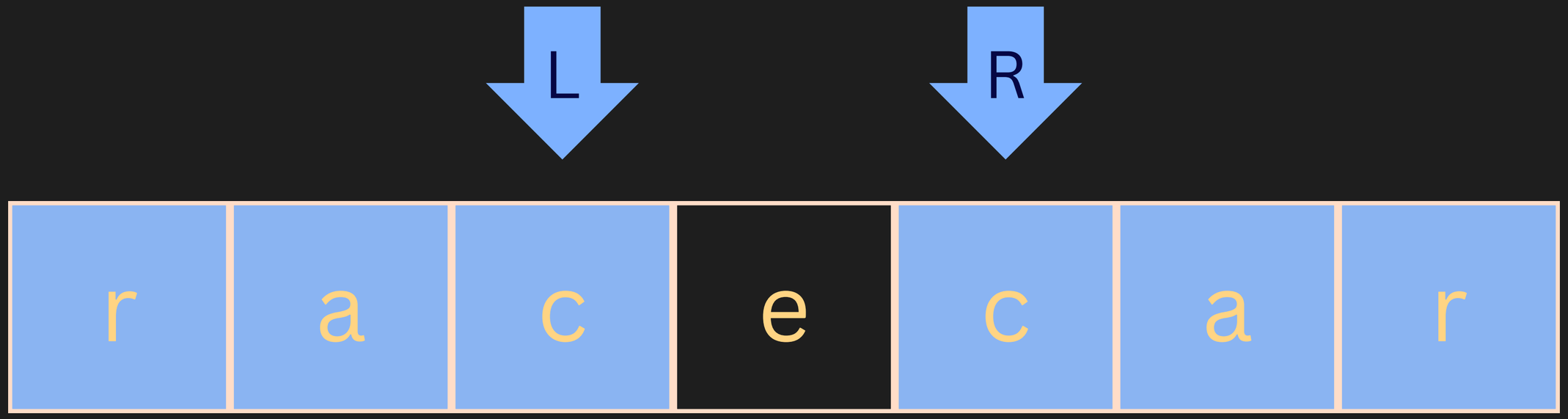
CASE STUDY 3: VALID PALINDROME

4. If the characters are different, the string is not a palindrome.
5. If the characters are the same move the pointers toward the center.
6. Repeat until the pointers meet.



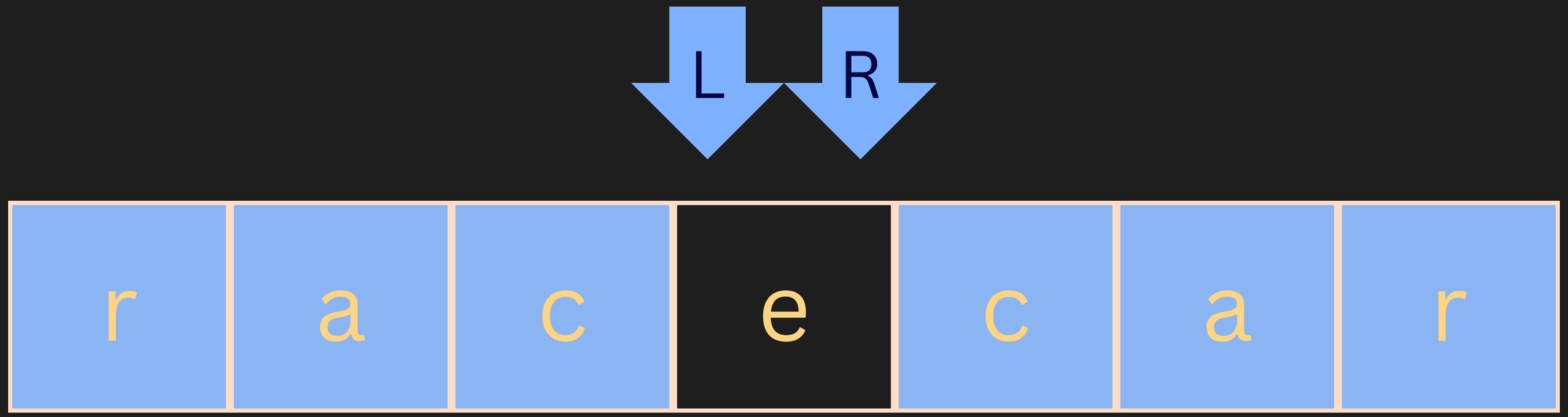
CASE STUDY 3: VALID PALINDROME

4. If the characters are different, the string is not a palindrome.
5. If the characters are the same move the pointers toward the center.
6. Repeat until the pointers meet.



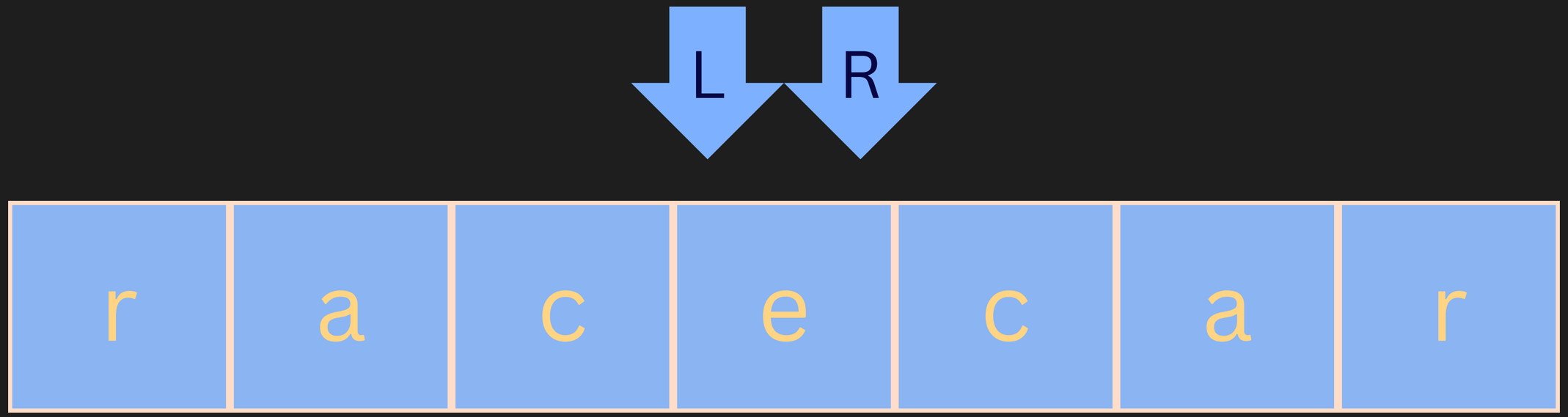
CASE STUDY 3: VALID PALINDROME

4. If the characters are different, the string is not a palindrome.
5. If the characters are the same move the pointers toward the center.
6. Repeat until the pointers meet.

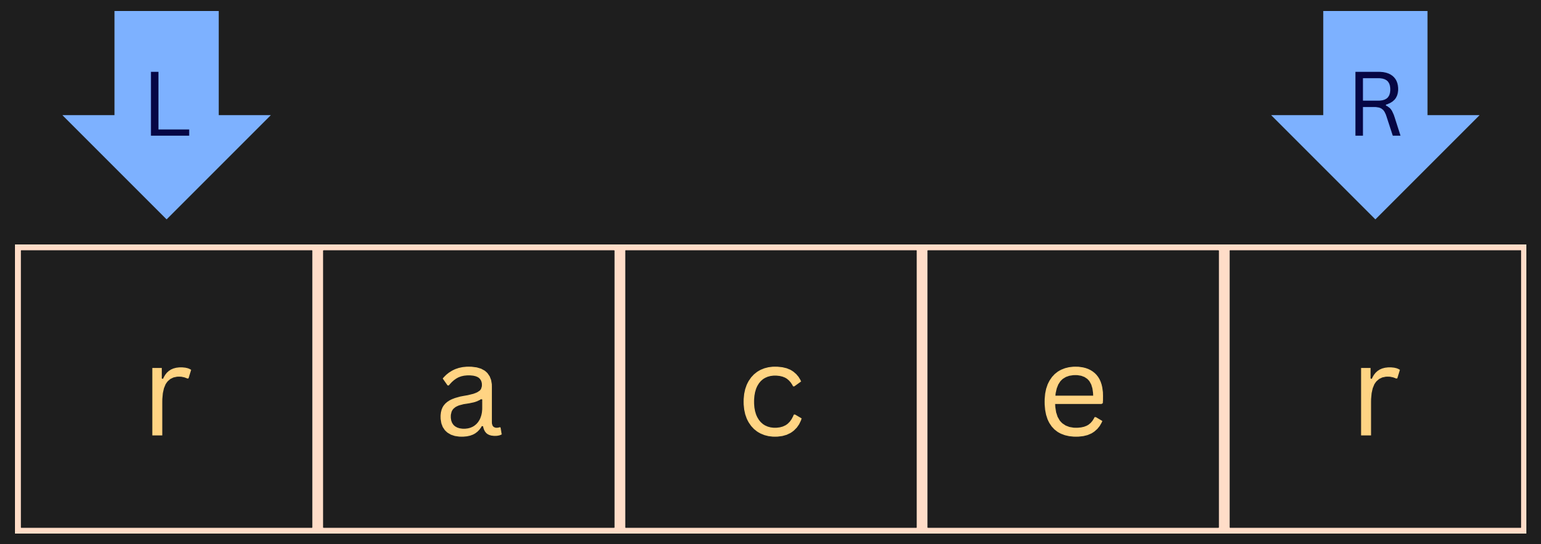


CASE STUDY 3: VALID PALINDROME

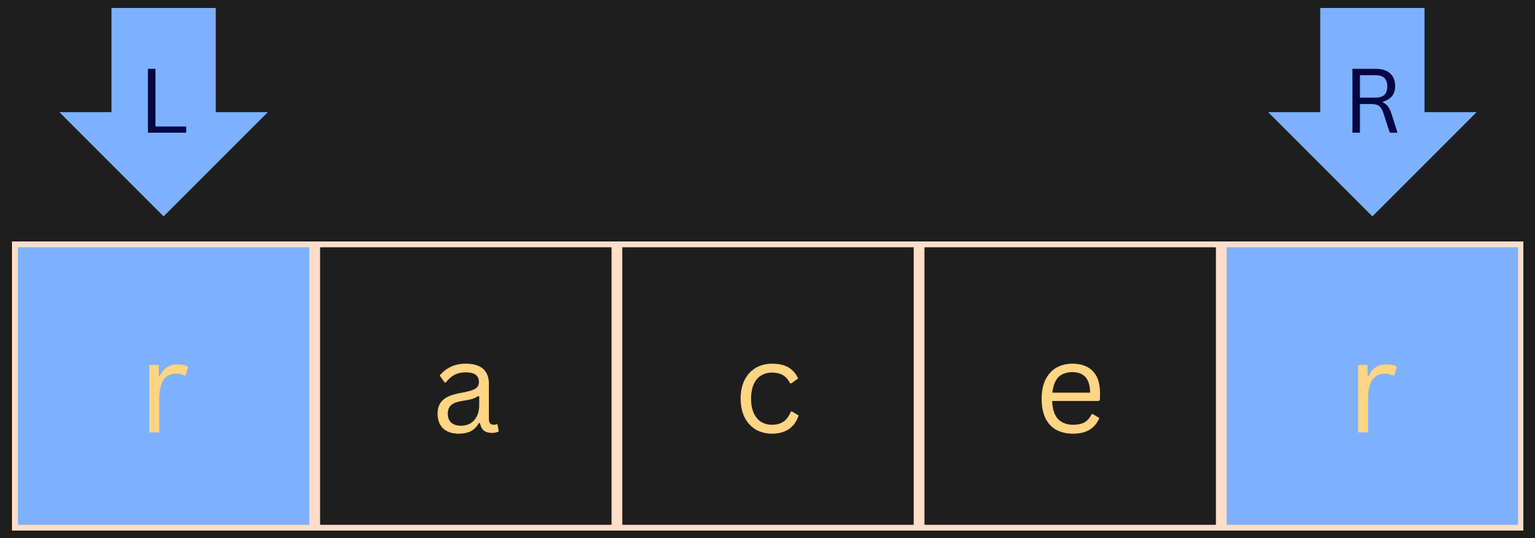
7. If all characters match, the string is a palindrome.



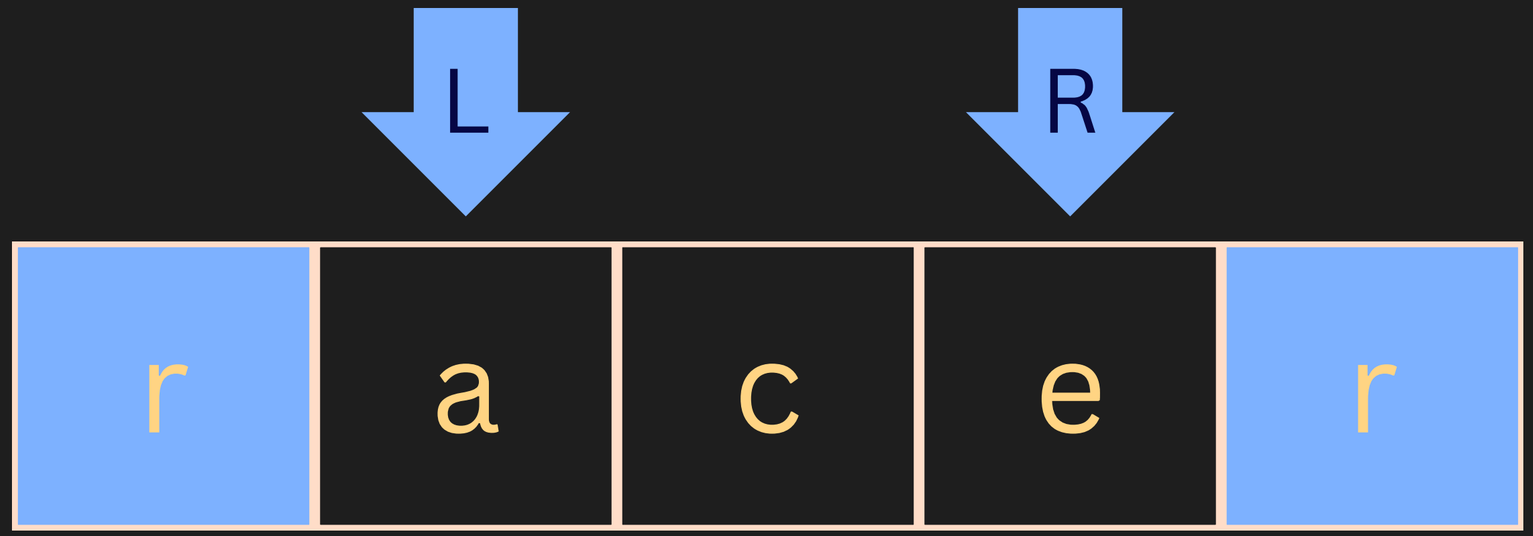
CASE STUDY 3: VALID PALINDROME



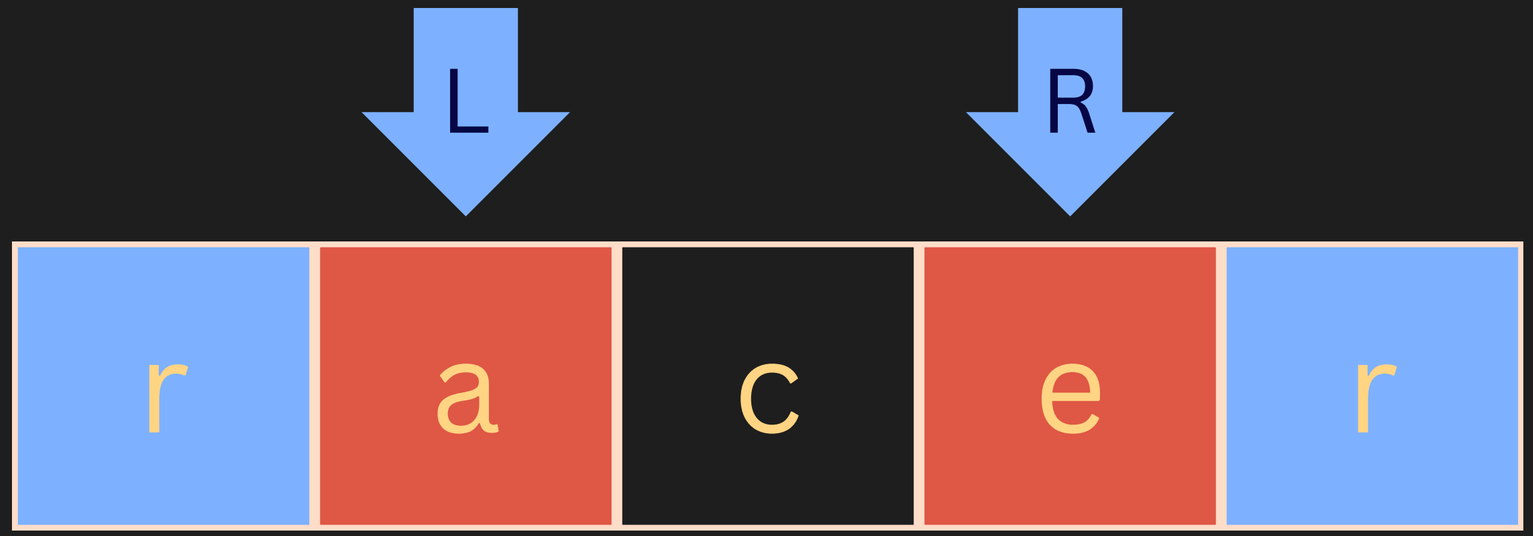
CASE STUDY 3: VALID PALINDROME



CASE STUDY 3: VALID PALINDROME



CASE STUDY 3: VALID PALINDROME



CASE STUDY 3: VALID PALINDROME

SOLUTION

```python
def is_palindrome(s):
    s = s.replace(" ", "").lower()

    left = 0
    right = len(s) - 1

    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1

    return True
```

FINAL CHALLENGE

Problem Statement

Given an array of strings and a target integer, find two **palindrome** strings from the array whose lengths sum up to the target value.

Return the two palindrome strings that satisfy this condition. If no such pair exists, return None.

Components:

Input: strings = ["racecar", "hello", "level", "world", "madam", "noon"]

target = 12

Output: "racecar, level"

Explanation:

- Palindromes: ["racecar", "level", "madam", "noon"]
- Lengths: [7, 5, 5, 4]
- racecar (7) + madam (5) = 12 ✓

FINAL CHALLENGE

Problem Statement

Given an array of strings and a target integer, find two **palindrome** strings from the array whose lengths sum up to the target value.

Return the two palindrome strings that satisfy this condition. If no such pair exists, return None.

Components:

Input: strings = ["abc", "def", "ghi"]

target = 10

Output: None

Explanation: No palindromes exist in the array

FINAL CHALLENGE

Problem Statement

Given an array of strings and a target integer, find two **palindrome** strings from the array whose lengths sum up to the target value.

Return the two palindrome strings that satisfy this condition. If no such pair exists, return None.

Components:

Input: strings = ["mom", "dad", "sister", "noon"]

target = 7

Output: "mom, noon"

Explanation: Palindromes: ["mom", "dad", "noon"]

- Lengths: [3, 3, 4]- mom (3) + noon (4) = 7 ✓

FINAL CHALLENGE

Code Template:

```python
def twoSum(nums, target):
    pass


def is_palindrome(s):
    pass


def main():
    strings = ["racecar", "hello", "level", "world", "madam", "noon"]
    target = 12

    # Insert Code Here

main()
```

FINAL CHALLENGE

Hints:

1. First, filter the array to keep only palindrome strings
2. Store the lengths of these palindromes in a separate array
3. Use the two-sum algorithm with a hashmap or brute force method to find two lengths that sum to the target

THANK YOU